

BÁO CÁO ĐỀ TÀI

Đề tài: Giới thiệu phương pháp SAT cho bài toán
Tối ưu dây chuyền sản xuất có Robot (Robotic Assembly Line Balancing)

January 12, 2026

1 Tổng quan

Nội dung nghiên cứu dựa trên bài báo gốc: "*Optimization of time and energy in straight one-sided robotic assembly lines*".

Phạm vi nghiên cứu:

- Áp dụng phương pháp SAT (Satisfiability) để giải quyết bài toán.
- Mặc dù bài toán gốc hướng tới tối ưu đa mục tiêu (thời gian và năng lượng) nhưng do giới hạn về bộ dữ liệu thử nghiệm nên trong khuôn khổ thử nghiệm hiện tại, nhóm nghiên cứu tập trung vào **tối ưu đơn mục tiêu là Chu kỳ thời gian (Cycle Time - CT)**.
- Phương pháp đa mục tiêu đã được phát triển và thử nghiệm thành công.

2 Phương pháp Nghiên cứu

2.1 Công nghệ sử dụng

Trong nghiên cứu này, bài toán cân bằng dây chuyền lắp ráp có robot (Robotic Assembly Line Balancing Problem – RALB) được giải quyết bằng phương pháp SAT (Boolean Satisfiability). Các ràng buộc số học được mã hóa dưới dạng **Pseudo-Boolean constraints (PB)**.

Các công nghệ và công cụ chính được sử dụng bao gồm:

- **Ngôn ngữ lập trình:** Python
- **Thư viện SAT:** PySAT
- **Bộ giải SAT:** Glucose4
- **Kỹ thuật mã hóa ràng buộc:**
 - Cardinality Encoding (CardEnc) cho các ràng buộc “đúng một”
 - Pseudo-Boolean Encoding (PBEnc) cho các ràng buộc tổng thời gian
- **Chiến lược tối ưu:** Tìm kiếm nhị phân (Binary Search) kết hợp kiểm tra tính khả thi bằng SAT

Thay vì tối ưu trực tiếp một hàm mục tiêu số học, phương pháp này tiếp cận bài toán thông qua việc kiểm tra tính khả thi của các ngưỡng Cycle Time (CT) khác nhau bằng SAT.

2.2 Quy trình giải quyết bài toán

Quy trình giải thuật được thiết kế thành các giai đoạn rõ ràng, tương ứng trực tiếp với cấu trúc chương trình cài đặt.

2.2.1 Khởi tạo tham số và tiền xử lý dữ liệu

Đọc dữ liệu đầu vào Dữ liệu được đọc từ tệp văn bản dạng bảng, trong đó mỗi dòng tương ứng với một tác vụ (task), bao gồm:

- Danh sách các tác vụ kế tiếp (Successor)
- Thời gian thực hiện của tác vụ với từng robot

Từ dữ liệu này, chương trình xây dựng:

- Tập tác vụ: $J = \{1, \dots, n\}$
- Tập robot: $R = \{1, \dots, m\}$
- Số trạm làm việc: N_w
- Đồ thị ưu tiên (precedence graph) G

Ước lượng cận dưới (Lower Bound – LB) Thời gian xử lý tối thiểu của mỗi tác vụ được xác định như sau:

$$T_j^{\min} = \min_{r \in R} T_{jr}$$

Sắp xếp các giá trị $T_j^{\min}$ theo thứ tự giảm dần và áp dụng nguyên tắc phân bổ đều lên các trạm, cận dưới của Cycle Time được tính bởi:

$$LB = \max_{k=1, \dots, n} \left\lceil \frac{\sum_{i=1}^k T_{(i)}^{\min}}{\left\lceil \frac{k}{N_w} \right\rceil} \right\rceil$$

Cách ước lượng này đảm bảo:

- LB không nhỏ hơn thời gian của tác vụ dài nhất
- Phản ánh tải tối thiểu có thể đạt được trong phân công

Ước lượng cận trên (Upper Bound – UB) Cận trên được tính dựa trên đường đi dài nhất trong đồ thị precedence, sử dụng thuật toán topo-sort kết hợp quy hoạch động:

$$UB = \max_{j \in J} \left(dist(j) + T_j^{\min} \right)$$

trong đó $dist(j)$ là tổng thời gian lớn nhất để hoàn thành tất cả các tác vụ tiền nhiệm của j .

Giá trị UB luôn được đảm bảo thỏa mãn $UB \geq LB$.

2.2.2 Mô hình hóa SAT

Biến quyết định Ba loại biến Boolean được sử dụng:

- x_{js} : tác vụ j được gán vào trạm s
- y_{sr} : trạm s sử dụng robot r
- z_{jsr} : tác vụ j tại trạm s được thực hiện bởi robot r

Ràng buộc gán Mỗi tác vụ phải được gán vào đúng một trạm:

$$\sum_{s=1}^{N_w} x_{js} = 1 \quad \forall j \in J$$

Mỗi trạm phải sử dụng đúng một robot:

$$\sum_{r=1}^m y_{sr} = 1 \quad \forall s = 1, \dots, N_w$$

Các ràng buộc này được mã hóa bằng Cardinality Encoding (equals-1).

Ràng buộc thứ tự (Precedence Constraints) Với mỗi quan hệ ưu tiên $u \rightarrow v$, trạm của u không được đứng sau trạm của v :

$$s_u \leq s_v$$

Trong SAT, điều này được mã hóa bằng các mệnh đề:

$$\neg x_{u,s_u} \vee \neg x_{v,s_v} \quad \forall s_v < s_u$$

Ràng buộc liên kết biến Biến z_{jsr} được định nghĩa như sau:

$$z_{jsr} \leftrightarrow (x_{js} \wedge y_{sr})$$

Được mã hóa bằng ba mệnh đề CNF:

$$\neg z_{jsr} \vee x_{js}$$

$$\neg z_{jsr} \vee y_{sr}$$

$$\neg x_{js} \vee \neg y_{sr} \vee z_{jsr}$$

2.2.3 Giải thuật tối ưu Cycle Time

Chiến lược Binary Search Cycle Time tối ưu được tìm bằng tìm kiếm nhị phân trên khoảng $[LB, UB]$. Tại mỗi bước, với một giá trị K , bài toán được kiểm tra tính khả thi bằng SAT.

Ràng buộc Cycle Time Với mỗi trạm s , tổng thời gian xử lý không được vượt quá K :

$$\sum_{j \in J} \sum_{r \in R} T_{jr} \cdot z_{jsr} \leq K$$

Ràng buộc này được mã hóa bằng PBEnc.l eq.

Guarded Constraints và Assumptions Mỗi ràng buộc PB được gán với một biến chọn σ_K :

$$\sigma_K \Rightarrow \left(\sum T_{jr} \cdot z_{jsr} \leq K \right)$$

Solver được gọi với assumption σ_K , cho phép:

- Tái sử dụng solver
- Giảm chi phí khởi tạo
- Tăng hiệu quả tìm kiếm

2.2.4 Trích xuất nghiệm

Nếu bài toán SAT thỏa mãn tại K :

- K được lưu làm nghiệm tốt nhất
- Tiếp tục tìm giá trị nhỏ hơn

Sau khi kết thúc tìm kiếm nhị phân, nghiệm SAT được giải mã thành:

- Trạm của mỗi tác vụ
- Robot tương ứng
- Tổng tải tại mỗi trạm

3 Quá trình Phát triển và Kết quả

3.1 Quá trình phát triển

- Dựa trên bài báo gốc, nhóm nghiên cứu đã liên tục cài đặt, thử nghiệm và cải tiến nhiều phiên bản mã nguồn và kết quả các giai đoạn thử nghiệm được lưu trong Github: <https://github.com/Xipog77/RALB>
- Hiện tại, chương trình đã được hoàn thiện ở mức tối ưu nhất có thể với phương pháp SAT.
- Tốc độ giải thuật của phiên bản hiện tại đã **tiệm cận** với phương pháp ILP/MIP ở quy mô bài toán nhỏ.

3.2 Kết quả thực nghiệm

Quá trình chạy thử nghiệm được thực hiện trên môi trường Google Colab với cấu hình giới hạn:

- Tổng thời gian triển khai: 30 phút.
- Time-out cho mỗi bài toán (Instance): 60 giây.
- Bộ dữ liệu: `borba-ritt-2018` (840 files).

3.2.1 So sánh sơ bộ

- **Với Dataset1 và Dataset2 (Bài toán nhỏ):** Phương pháp SAT cho ra kết quả chính xác, giống với phương pháp MIP, tuy nhiên thời gian thực thi vẫn có phần chậm hơn.
- **Với Dataset Borba-Ritt-2018 (Bài toán lớn):** Dưới đây là bảng kết quả mẫu từ phương pháp MIP (để đối chứng):

3.2.2 Bảng so sánh kết quả

Bảng 1 dưới đây minh họa rõ sự chênh lệch hiệu năng giữa hai phương pháp đối với các bài toán quy mô lớn (100 tasks).

Table 1: So sánh hiệu năng MIP vs SAT trên dataset Borba-Ritt (100 Tasks)

Instance (File) (Tasks=100)	MIP (SCIP)		SAT (Glucose4)	
	Trạng thái	Cycle Time	Trạng thái	Cycle Time
100_BN_2_bimodal_1_14_2.alb	FEASIBLE	16495.0	TIMEOUT	
100_BN_2_bimodal_1_14_5.alb	FEASIBLE	46202.0	TIMEOUT	
100_BN_2_bimodal_1_25_2.alb	FEASIBLE	23932.0	TIMEOUT	
100_BN_2_bottom_1_14_2.alb	OPTIMAL	27008.0	TIMEOUT	
100_BN_2_bottom_2_14_2.alb	FEASIBLE	6525.0	TIMEOUT	

3.3 Nhận xét và Đánh giá

- **Ưu điểm:** Phương pháp SAT mang lại một hướng tiếp cận hiện đại cho bài toán cân bằng dây chuyền. Điểm mạnh cốt lõi là khả năng tận dụng các bộ giải SAT Solvers giúp giải quyết nhanh các bài toán kiểm tra tính khả thi ở quy mô nhỏ và trung bình.
- **Hạn chế:** Phương pháp này bộc lộ nhược điểm nghiêm trọng khi xử lý các bộ dữ liệu quy mô lớn (như bộ 100 tasks của Borba-Ritt). Các vấn đề chính bao gồm:
 - **Bùng nổ số lượng mệnh đề (Clause Explosion):** Khác với MIP sử dụng trực tiếp các biến số nguyên (integer variables) để biểu diễn thời gian, SAT buộc phải mã hóa (encoding) các giá trị số thành chuỗi các biến nhị phân và mệnh đề CNF. Điều này làm kích thước bài toán tăng vọt theo cấp số nhân, dẫn đến tiêu tốn tài nguyên bộ nhớ (RAM) khổng lồ.
 - **Hiệu suất tối ưu hóa:** Chiến lược tìm kiếm nhị phân (Binary Search) yêu cầu gọi lại bộ giải SAT nhiều lần liên tiếp. Với không gian tìm kiếm lớn và số lượng biến nhị phân dày đặc, quá trình này dễ dẫn đến hiện tượng tràn bộ nhớ hoặc vượt quá giới hạn thời gian (Timeout) trước khi tìm ra giá trị Cycle Time tối ưu, khiến hiệu suất tổng thể kém hơn so với các mô hình MIP truyền thống trên các tập dữ liệu lớn.

4 Hướng nghiên cứu trong tương lai

Nhóm nghiên cứu dự kiến sẽ tiếp tục:

1. Cải thiện khả năng tính toán của thuật toán để xử lý các bộ dữ liệu lớn hơn.
2. Nghiên cứu các phương pháp mã hóa (Encoding) hiệu quả hơn để giảm thiểu số lượng mệnh đề SAT.
3. Kết hợp các kỹ thuật Heuristic để hỗ trợ tìm kiếm cận tốt hơn cho quá trình giải.